Time Complexity

The time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input.

The time required by the algorithm to solve given problem is called ***time complexity*** of the algorithm.

No	Description	Complexity
Rule 1	Any assignment statements and if statements that are executed once regardless of the size of the problem	O(1)
Rule 2	A simple "for" loop from 0 to n (with no internal loops)	O(n)
Rule 3	A nested loop of the same type takes quadratic time complexity	$O(n^2)$
Rule 4	A loop, in which the controlling parameter is divided by two at each step	O(log n)
Rule 5	When dealing with multiple statements, just add them up	

Ex1:-

```
def multiply_numbers(n):
        return n*n
```

Time complexity → O(1)

Ex2:-

```
def print_items(n):
    for i in range(n):
        print(i)
```

Time complexity → O(N)

Ex3:-

```
def print_items(n):
    for i in range(n):
        for j in range(n):
            print(i, j)
```

Time complexity → $O(N^2)$

Ex4:-

```
def print_items(n):
    for i in range(n):
        for j in range(n):
            print(i, j)
    for k in range(n):
        print(k)
```

Time complexity → $O(N^2) + O(N) = O(N^2)$

Ex5:-

```
def findBiggestNumber(sampleArray):
    biggestNumber = sampleArray[0] ----------------------------------------------> O(1)
    for index in range(1,len(sampleArray)):---------------------->O(n)
        if sampleArray[index] > biggestNumber: ------> O(1)
            biggestNumber = sampleArray[index] ------> O(1)
    print(biggestNumber) ---------------------------------------------------------> O(1)
```

Time complexity → O(1) + O(N) + O(1) + O(1) + O(1) = O(N)

Ex6:-

What is the time complexity of the following Python code snippet?

```
sum = 0
for i in range(n):
    sum += i
```

Time complexity → O(N)

Ex7:-

What is the time complexity of the following Python code snippet?

```
sum = 0
i = 1
while i <= n:
    sum += i
    i *= 2
```

Time complexity → O(log n)

The loop iterates 'log n' times, as 'i' is multiplied by 2 in each iteration. The time complexity is O(log n).

Logarithmic time complexity is denoted as **O(log n).** It is a measure of how the runtime of an algorithm scales as the input size increases.

Notation	Complexity
O(1)	Constant
O(log N)	Logarithmic
O(N)	Linear time
O(N * log N)	Log linear
O(N^2)	Quadratic

Recursion

- **A way of solving a problem by having a function calling itself.**
- **Performing the same operation multiple times with different inputs.**

- In every step we try smaller inputs to make the problem smaller.

- Base condition is needed to stop recursion otherwise infinite loop will occur.

Why recursion?

a. Recursive thinking is really important in programming and it helps you break big problems in smaller ones and easier to use.

b. The prominent use of recursion in data structures like Trees and Graphs.

c. **Used in many algorithms like divide and conquer, greedy and dynamic programming.**

Ques:

Factorial using recursion

```
def factorial(n):
  """
  Calculates the factorial of a non-negative integer using recursion.
  """
  # Base case: when n is 0 or 1, the factorial is 1. This stops the recursion.
  if n == 0 or n == 1:
    return 1
  # Recursive case: n! = n * (n-1)!
  else:
    return n * factorial(n - 1)

# Example Usage:
num = 5
if num < 0:
  print("Factorial is not defined for negative numbers.")
else:
  result = factorial(num)
  print(f"The factorial of {num} is {result}")
```

Ques:

Fib series using recursion

```python
def fibonacci(n):
    """
    Calculates the nth Fibonacci number recursively.
    """
    # Base case: handles the first two terms
    if n <= 1:
        return n
    else:
        # Recursive step: sum of the two preceding terms
        return fibonacci(n-1) + fibonacci(n-2)

# Driver code to print the series
if __name__ == "__main__":
    # Number of terms to display
    nterms = 10

    # Check if the number of terms is valid
    if nterms <= 0:
        print("Please enter a positive integer")
    else:
        print("Fibonacci sequence:")
        # Loop through the number of terms and print each one
        for i in range(nterms):
            print(fibonacci(i))
```

Ques:

Sum of Elements in a List

```python
def sum_list(numbers):
    """
    Calculates the sum of all elements in a list recursively.
    """
    # Base case: An empty list has a sum of 0
    if len(numbers) == 0:
        return 0
    # Recursive case: Add the first element to the sum of the remaining list
    else:
        return numbers[0] + sum_list(numbers[1:])

# Test the function
my_list = [1, 2, 3, 4, 5]
print(f"The sum of the list {my_list} is {sum_list(my_list)}")
# Output: The sum of the list [1, 2, 3, 4, 5] is 15
```